Introduction to Assembly Language

Computer Organization and Assembly Language

Computer Science Department

National University of Computer and Emerging Sciences Islamabad

Instruction Execution and Straight-Line Sequencing

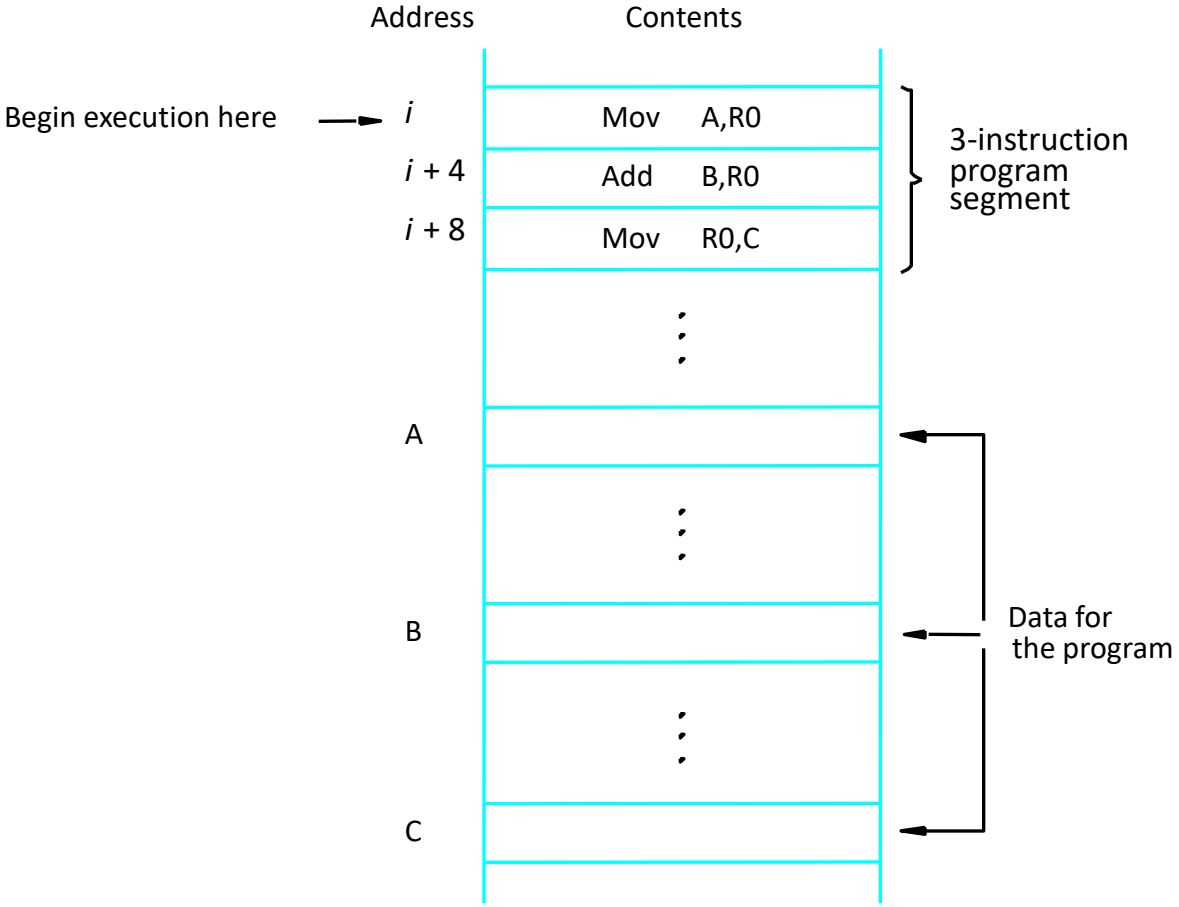
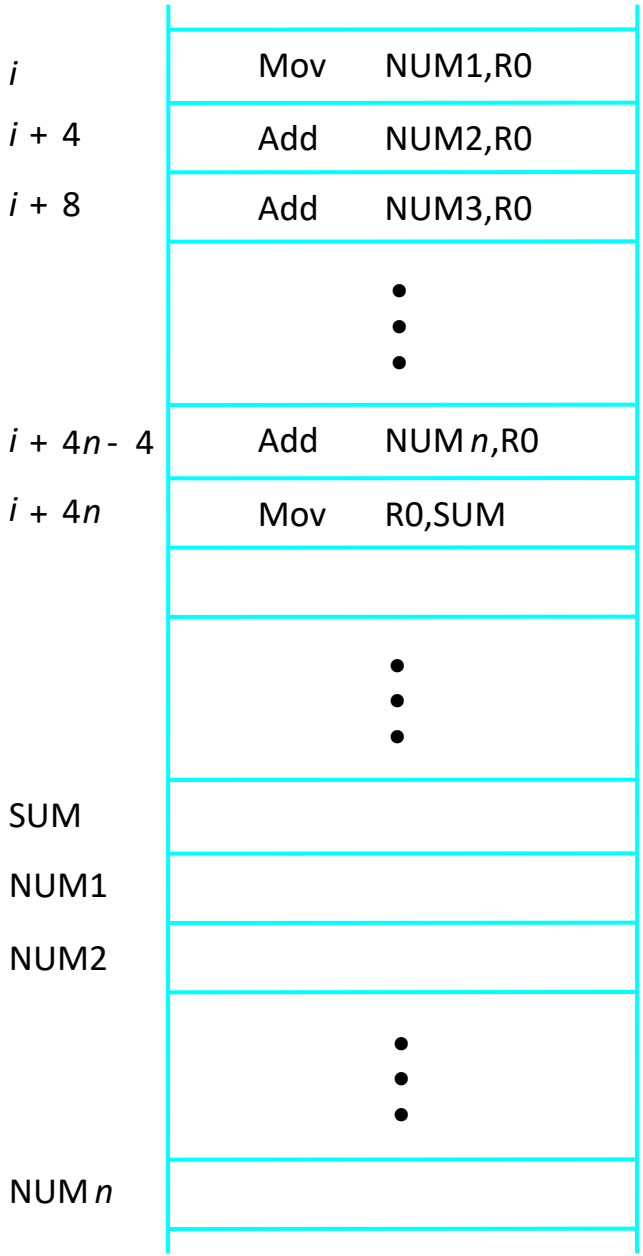


Figure 2.8. A program for C ← [A] + [B].

Assumptions:
- One memory operand per instruction
- 32-bit word length
- Memory is byte addressable
- Full memory address can be directly specified in a single-word instruction

Two-phase procedure
-Instruction fetch
-Instruction execute



What is the maximum memory capacity that 8086 processor can address?

- Up to 1MB (1024kb) of data
- Each memory location is identified by a 20-bit Physical address

Figure 2.9. A straight-line program for adding *n* numbers.

Assembly Language

Assembly Language Programming

- For Intel Systems
- Software Required
  - MASM
  - Visual Studio

Format of Program

- Identifiers are case insensitive.
- Comments follow ';' character.
- Sections: **Data, Code**, and Stack.
- Hexadecimal Numbers have 'h' suffix.

Basics

- Directives vs. Instructions.
  - Use Directives to tell assembler what to do.
  - Use Instructions to tell CPU what to do.

- Procedure defined by:
  - [Name] PROC
  - [Name] ENDP
- Instruction Format:
  - LABEL (optional), Mnemonic, Operands

Directives

- Commands that are recognized and acted upon by the assembler
  - Not part of the Intel instruction set
  - Used to declare code, data areas, select memory model, declare procedures, etc.
- Different assemblers have different directives
  - NASM != MASM, for example

Integer Literals

- An *integer literal* (also known as an *integer constant*) is made up of an optional leading sign, one or more digits, and an optional radix character that indicates the number's base:

```
[{+ | - }] digits [ radix ]
```

h	hexadecimal	r	encoded real
q/o	octal	t	decimal (alternate)
d	decimal	y	binary (alternate)
b	binary		

TABLE 3-1 Arithmetic Operators.

Operator	Name	Precedence Level
( )	Parentheses	1
+, −	Unary plus, minus	2
*, /	Multiply, divide	3
MOD	Modulus	3
+, −	Add, subtract	4

- **Real Number Literals**

  `[sign] integer.[integer][exponent]`

- Examples
  - 2., +3.0, -44.2E+05, 26.E5

- A **character literal** is a single character enclosed in single or double quotes. The assembler stores the value in memory as the character's binary ASCII code.

  - E.g. 'A', "x".

- A **string literal** is a sequence of characters (including spaces) enclosed in single or double quotes.
  - E.g. 'ABC‘, 'X‘, “Good night, Gracie“

- **Reserved words** have special meaning and can only be used in their correct context (not case sensitive).
  - Instruction mnemonics, Register names, Directives
  - Attributes, which provide size and usage information for variables and operands. Examples are BYTE and WORD
  - Operators, used in constant expressions
  - Predefined symbols, such as @data, which return constant integer values at assembly time.

IDENTIFIER

- An **identifier** is a programmer-chosen name. It might identify a variable, a constant, a procedure, or a code label.
- Rules:
  - They may contain between 1 and 247 characters.
  - They are not case sensitive.
  - The first character must be a letter (A..Z, a..z), underscore (_), @ , ?, or $. Subsequent characters may also be digits.
  - An identifier cannot be the same as an assembler reserved word.

Instructions

- Assembled into machine code by assembler
- Executed at runtime by the CPU
- Member of the Intel IA-32 instruction set
- Parts
  - Label
  - Mnemonic
  - Operand
  - Comment

Directive Vs Instruction

- **`myVar DWORD 26`**

DWORD directive tells the assembler to reserve space in the program for a doubleword variable.

`mov eax,myVar`

The MOV instruction, on the other hand, executes at runtime, copying the contents of myVar to the EAX register

I/O

- Not as easy as you think, if you program it yourself.
- We can use the library provided by the author of the textbook.
- But we will do it ourself.

Program/Data Representations

- Consider this simple view: The whole program is stored in main memory.
  - Including program instructions (code) and data.
  - CPU loads the instructions and data from memory for execution.
  - Don't worry about the disk for now.

Data Transfer Instructions

- MOV is for moving data between:
  - Memory
  - Register
  - Immediate (constant)
- Almost all combinations, except:
  - Memory to Memory!

MOV Instruction

- Move from source to destination. Syntax:

  MOV *destination,source*

- No more than one memory operand permitted
- Both operands must be the same size.
- CS, EIP, and IP cannot be the destination
- No immediate to segment moves

```
.data
count BYTE 100
wVal  WORD 2
.code
    mov bl,count
    mov ax,wVal
    mov count,al

    mov al,wVal                 ; error
    mov ax,count                ; error
    mov eax,count               ; error
```

YOUR TURN . . .

Explain why each of the following MOV statements are invalid:

```
.data
bVal   BYTE   100
bVal2  BYTE   ?
wVal   WORD   2
dVal   DWORD  5
.code
   mov ds,45                ; a.
   mov esi,wVal             ; b.
   mov eip,dVal             ; c.
   mov 25,bVal              ; d.
   mov bVal2,bVal           ; e.
```

Food for Thought

- Example: MOV AX, 25
- But, where is the number stored? In memory?
- A related question: where is the instruction (MOV AX, 25) stored?
- What do we do if we want to move the data in memory address 25 to AX?

Memory to Memory?

- Must go through a register…

```
.data
Var1 WORD 100h
Var2 WORD ?
.code
    MOV AX, var1
    MOV var2, AX
```

Direct Memory Operands

- A direct memory operand is a named reference to storage in memory
- The named reference (label) is automatically dereferenced by the assembler

```
.data
var1 BYTE 10h
.code
mov al,var1          ; AL = 10h
mov al,[00010400]    ; if var1 at offset 10400h
mov al,[var1]        ; AL = 10h
```

alternate format

Character String

- So how are strings like “Hello, World!” are stored in memory?
- ASCII Code! (or Unicode…etc.)
- Each character is stored as a byte.
- Review: how is “1234” stored in memory?

Integer

- A byte can hold an integer number:
  - between 0 and 255 (unsigned) or
  - between –128 and 127 (2's compliment)
- How to store a bigger number?
- Review: how is 1234 stored in memory?

3.2 EXAMPLE

```
 1: ; AddTwo.asm - adds two 32-bit integers
 2: ; Chapter 3 example
 3:
 4: .386
 5: .model flat,stdcall
 6: .stack 4096
 7: ExitProcess PROTO, dwExitCode:DWORD
 8:
 9: .code
10: main PROC
11:   mov eax,5 ; move 5 to the eax register
12:   add eax,6 ; add 6 to the eax register
13:
14: INVOKE ExitProcess,0
15: main ENDP
16: END main
```

- **`.386`** directive identifies the program as a 32-bit program that can access 32-bit registers and addresses.
- **`.model flat, stdcall`** selects the programs memory model, and identifies the calling convention.
- The **`stdcall`** keyword tells the assembler how to manage the runtime stack when procedures are called.
  - It is a **calling convention**, that is a scheme for how subroutines receive parameters from their caller and how they return a result.

MODEL DIRECTIVE

Memory Model	Size of Code	Size of Data
TINY	Code + Data < 64KB	Code + data < 64KB
SMALL	Less than 64KB	Less than 64KB
MEDIUM	Can be more than 64KB	Less than 64 KB
COMPACT	Less than 64KB	Can be more than 64KB
LARGE*	Can be more than 64K	Can be more than 64KB
HUGE**	Can be more than 64K	Can be more than 64KB

- **`.stack 4096`** sets aside 4096 bytes of storage for the runtime stack.
- It tells the assembler how many bytes of memory to reserve for the program's runtime stack.

- Stack are used :

1. to hold passed parameters
2. to hold the address of the code that called the function. The CPU uses this address to return when the function call finishes, back to the spot where the function was called.
3. Stack can hold local variables.

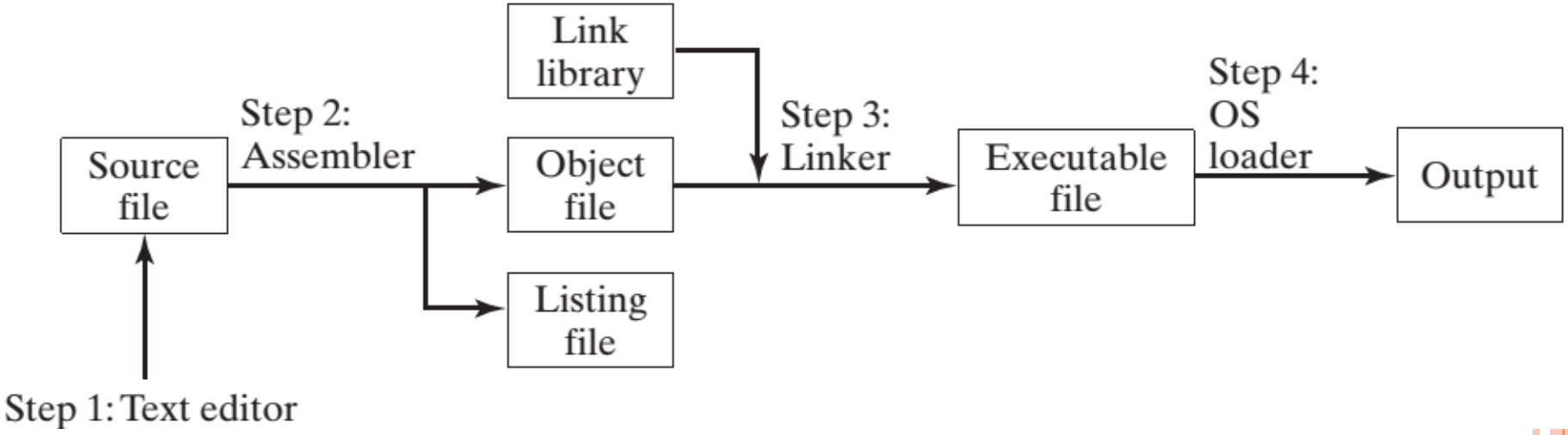
- **`.code`** marks the beginning of the code area of a program.

- When your program is ready to finish, it calls **`ExitProcess`** that returns an integer that tells the operating system that your program worked just fine.

- The **`ENDP`** directive marks the end of a procedure.
  - Our program had a procedure named main, so the endp must use the same name.

- Line 16 uses the **END** directive to mark the last line to be assembled (end of program), and it identifies the program entry point (main).
  - If you add any more lines to a program after the END directive, they will be ignored by the assembler.

Assembling, Linking, and Running Programs



```
 1:    ; AddTwo.asm - adds two 32-bit integers.
 2:    ; Chapter 3 example
 3:
 4:    .386
 5:    .model flat,stdcall
 6:    .stack 4096
 7:    ExitProcess PROTO,dwExitCode:DWORD
 8:
 9:    00000000                        .code
10:    00000000                        main PROC
11:    00000000  B8 00000005              mov  eax,5
12:    00000005  83 C0 06                 add  eax,6
13:
14:                                       invoke ExitProcess,0
15:    00000008  6A 00                    push   +000000000h
16:    0000000A  E8 00000000 E            call   ExitProcess
17:    0000000F                        main ENDP
18:                                    END main
```

Defining data

- The assembler recognizes a basic set of ***intrinsic data types***, which describe types in terms of their size.

```
[name] directive initializer [, initializer]...
```

- ***Initializer***: At least one *initializer* is required in a data definition, even if it is zero.

```
.data
  sum DWORD 0
```

- If you prefer to leave the variable uninitialized (assigned a random value), the **?** symbol can be used as the initializer

```
sum DWORD ?
```

Type	Usage
BYTE	8-bit unsigned integer. B stands for byte
SBYTE	8-bit signed integer. S stands for signed
WORD	16-bit unsigned integer
SWORD	16-bit signed integer
DWORD	32-bit unsigned integer. D stands for double
SDWORD	32-bit signed integer. SD stands for signed double
FWORD	48-bit integer (Far pointer in protected mode)
QWORD	64-bit integer. Q stands for quad
TBYTE	80-bit (10-byte) integer. T stands for Ten-byte
REAL4	32-bit (4-byte) IEEE short real
REAL8	64-bit (8-byte) IEEE long real
REAL10	80-bit (10-byte) IEEE extended real

Table 3-3 Legacy Data Directives.

Directive	Usage
DB	8-bit integer
DW	16-bit integer
DD	32-bit integer or real
DQ	64-bit integer or real
DT	define 80-bit (10-byte) integer

Defining BYTE and SBYTE Data

- Each initializer must fit into 8 bits of storage

```
.data
value1 BYTE 'A'          ; character constant
value2 BYTE 0            ; smallest unsigned byte
value3 BYTE 255          ; largest unsigned byte
value4 SBYTE -128        ; smallest signed byte
value5 SBYTE +127        ; largest signed byte
value6 BYTE ?            ; uninitialized byte
```

Operand	Description
reg8	8-bit general-purpose register: AH, AL, BH, BL, CH, CL, DH, DL
reg16	16-bit general-purpose register: AX, BX, CX, DX, SI, DI, SP, BP
reg32	32-bit general-purpose register: EAX, EBX, ECX, EDX, ESI, EDI, ESP, EBP
reg	Any general-purpose register
sreg	16-bit segment register: CS, DS, SS, ES, FS, GS
imm	8-, 16-, or 32-bit immediate value
imm8	8-bit immediate byte value
imm16	16-bit immediate word value
imm32	32-bit immediate doubleword value
reg/mem8	8-bit operand, which can be an 8-bit general register or memory byte
reg/mem16	16-bit operand, which can be a 16-bit general register or memory word
reg/mem32	32-bit operand, which can be a 32-bit general register or memory doubleword
mem	An 8-, 16-, or 32-bit memory operand

Multiple Initializers

- If multiple initializers are used in the same data definition, its label refers only to the offset of the first initializer.

	Offset	Value
list	0000	10
	0001	20
	0002	30
	0003	40

```
list BYTE 10,20,30,40
```

Within a single data definition, its initializers can use different radixes. Character and string literals can be freely mixed. In the following example, **list1** and **list2** have the same contents:

```
list1 BYTE 10, 32, 41h, 00100010b
list2 BYTE 0Ah, 20h, 'A', 22h
```

Examples that use multiple initializers:

```
.data
Array1 BYTE 10h,20h,?,40h
       BYTE 50h,60h,70h,80h
        WORD 81h,82h,83h
list3  BYTE ?,32,41h


.code
   Mov al, array1 + 7    ; al = 80h
```

	Offset	Value
array1	0000	10
	0001	20
	0002	
	0003	40
	0004	50
	0005	60
	0006	70
	0007	80
	0008	81
	0009	00
	000A	82
	000B	00
	000C	83
	000D	00
list3	000E	
	000F	32
	0010	41

- **Defining Strings**:
- The most common type of string ends with a null byte (containing 0), called a *null-terminated* string.

```
greeting1 BYTE "Good afternoon",0
greeting2 BYTE 'Good night',0
```

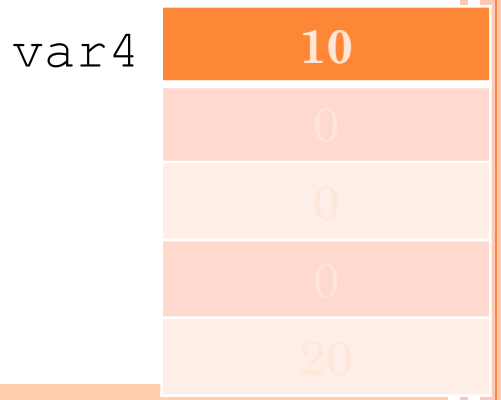
- Each character uses a byte of storage.
- The rule that byte values must be separated by commas does not apply on strings.

- The ***DUP operator*** allocates storage for multiple data items, using a integer expression as a counter

```
x BYTE 20 DUP(0)              ;20 bytes, all equal to zero
y BYTE 20 DUP(?)              ;20 bytes, uninitialized
S BYTE 4 DUP("STACK")         ;20 bytes: "STACKSTACKSTACKSTACK"

Z BYTE 7 Dup (2 Dup(17,19)); 17 19 17 19
```

```
var4 BYTE 10,3 DUP(0), 20
```



DUP directive is used to duplicate or replicate data values in an array or a block of memory

- **Defining WORD and SWORD Data**
- The WORD (define word) and SWORD (define signed word) directives create storage for one or more 16-bit integers:

```
word1   WORD   65535              ; largest unsigned value
word2   SWORD -32768              ; smallest signed value
word3   WORD   ?                  ; uninitialized, unsigned
word4   WORD   "AB"               ; double characters
myList  WORD   1,2,3,4,5          ; array of words
array   WORD   5 DUP(?)           ; uninitialized array
```

- The legacy DW directive can also be used:

```
val1 DW 65535 ; unsigned
val2 DW -32768 ; signed

myList WORD 1,2,3,4,5
```

Offset	Value
0000:	1
0002:	2
0004:	3
0006:	4
0008:	5

Defining DWORD and SDWORD Data

- Storage definitions for signed and unsigned 32-bit integers:

```
val1 DWORD  12345678h      ; unsigned
val2 SDWORD -2147483648    ; signed
val3 DWORD  20 DUP(?)      ; unsigned array
val4 SDWORD -3,-2,-1,0,1   ; signed array
```

```
myList DWORD 1,2,3,4,5
```

Offset	Value
0000:	1
0004:	2
0008:	3
000C:	4
0010:	5

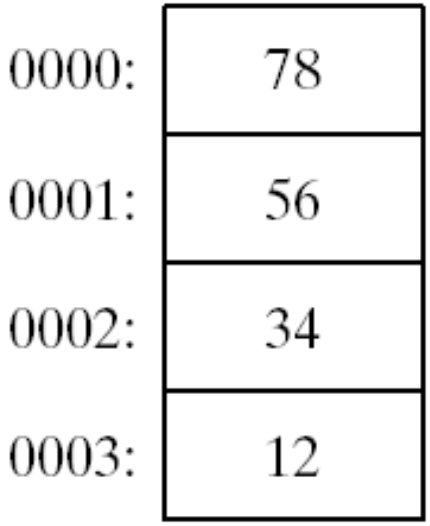
Labels in Assembly

- A label is an identifier that is followed by a colon.
- Names suffixed with colons (:) are symbolic labels.
- The labels do not create code.
- They are simply a way to tell the assembler that those locations have symbolic names.
- Example
  - Label1:
  - Above:
  - Start:

Little Endian Order

- All data types larger than a byte store their individual bytes in reverse order. The least significant byte occurs at the first (lowest) memory address.

```
val1 DWORD 12345678h
```



Declaring Unitialized Data

- **Declaring Uninitialized Data**
- Use the `.data?` directive to declare an uninitialized data segment:

```
.data?
```

- When defining a large block of uninitialized data, the `.DATA?` directive reduces the size of a compiled program
- Within the segment, declare variables with "?" initializers:

```
smallArray DWORD 10 DUP(?)
```

Advantage: the program's EXE file size is reduced.

```
.data
smallArray DWORD 10 DUP(0) ; 40 bytes
.data?
bigArray DWORD 5000 DUP(?) ; 20,000 bytes, not initialized
```

The following code, on the other hand, produces a compiled program 20,000 bytes larger:

```
.data
smallArray DWORD 10 DUP(0) ; 40 bytes
bigArray DWORD 5000 DUP(?) ; 20,000 bytes
```

3.5 Symbolic Constants

- A *symbolic constant* (or *symbol definition*) is created by associating an identifier (a symbol) with an integer expression or some text.
- Symbols do not reserve storage.

	Symbol	Variable
Uses storage?	No	Yes
Value changes at runtime?	No	Yes

- We used **EQU** and **TEXTEQU** directives to create symbols representing arbitrary text.
- **Equal-Sign Directive**

```
name = expression
```

  - expression is a 32-bit integer (expression or constant)
  - may be redefined
  - *name* is called a *symbolic constant*

- *Current Location Counter :* the symbol **$**, is the current location counter, that returns offset of current location:

```
selfPtr DWORD $
```

  - This statement declares a variable named **selfPtr** and initializes it with the variable's offset value.

- **Calculating the Sizes of Arrays and Strings**
- As $ operator returns the offset associated with the current program statement, it can be used to calculate the size of array/string:

```
list BYTE 10,20,30,40
ListSize = ($ - list)
```

- **ListSize** must follow immediately after **list.**

ListSize must follow immediately after list.

```
list BYTE 10,20,30,40
Var2 BYTE 20 DUP (?)
ListSize = ($ - list)
```

Rather than calculating the length of string manually let the assembler do it

How?

Overlapping Values

```
.data
OneByte BYTE 78H
oneWord Word 1234h
oneDword Dword 12345678h
.code

Mov eax,0      ;
Mov al,oneByte ;
Mov ax,oneWord  ;
Mov eax, oneDword
Mov ax,0
```

eax=12340000h

Overlapping Values

```
.data
        aValue DWORD 12345678h
        bValue DWORD 87654321h
.code
         Mov eax, aValue
         Mov ebx, bValue
         Mov ecx, 1234h
       Mov edx, 0
       Add eax, ebx
       Sub eax, ecx
```

Reference

- Assembly language for Intel Base Cmputers
- Kip R Irvine
  - Chapter 2,3

That's it for today!

Examples

EXAMPLE

```
.data
        num1 dw 5
        num2 dw 10
        num3 dw 15
        num4 dw 0
.code
```

add three numbers, num1, num2, and num3, and stores the result in a variable named num4

EXAMPLE

```
.data
        num1 dw 5
        num2 dw 10
        num3 dw 15
        num4 dw 0
.code
        mov ax, [num1]
        mov bx,[num2]
        add ax,bx
        mov bx, [num3]
        add ax,bx
        mov [num4], ax
```

EXAMPLE

```
.data
        num1 dw 5
             dw 10
             dw 15
             dw 0
.code
        mov ax, [num1]
        mov bx,[num+2]
        add ax,bx
        mov bx, [num1+4]
        add ax,bx
        mov [num1+6], ax
```

EXAMPLE

```
.data
        num1 db 5
             db 10
             db 15
             db 0
.code
        mov al, [num1]
        mov bl,[num+1]
        add al,bl
        mov bl, [num1+2]
        add al,bl
        mov [num1+3], al
```

EXAMPLE

```
.data
        num1 dd 5,10,15,0
.code
        mov ebx, offset num1
        mov eax,[ebx]
        add ebx,4
        add eax,[ebx]
        add ebx,4
        add eax,[ebx]
        add ebx,4
        mov [ebx],eax
```